Custom Gecko Browser: Systems Architecture & Implementation Plan

Engineering Blueprint for a High-Fluidity, Low-RAM Rust/C++ Browser with Built-In uBlock Origin, Free Cloudflare WARP WireGuard VPN, and Universal Dark Mode

1. Diagnostic Baseline: Why Opera GX Uses 1,351.6 MB Despite Efficiency Mode

Your Windows Task Manager captures expose the fundamental flaw in how mainstream browsers implement resource savers. In your first capture, **Opera GX** consumes **1,351.6 MB of RAM** and **1.4% CPU** across **22 subprocesses**. Even though 13 of those subprocesses display the green leaf for Windows Efficiency Mode (EcoQoS), those 13 throttled background processes still hold **767.1 MB of committed RAM** (56.8% of the browser's entire footprint), led by a single background tab holding 240.4 MB and main UI/GPU processes holding 293.2 MB and 131.1 MB.

Why does this happen? Because Windows `PROCESS_POWER_THROTTLING_EXECUTION_SPEED` only instructs the kernel scheduler to run threads at lower clock frequencies and prefer efficiency cores—it never triggers garbage collection, never unloads inactive DOM trees, and never trims the process working set. By contrast, your **Firefox (Gecko)** capture runs **10 subprocesses** at **617.8 MB of RAM** (a 54.3% memory reduction out of the box) and **0.5% CPU**, with 494.2 MB across 9 active processes and 123.7 MB in 1 Efficiency Mode content process.

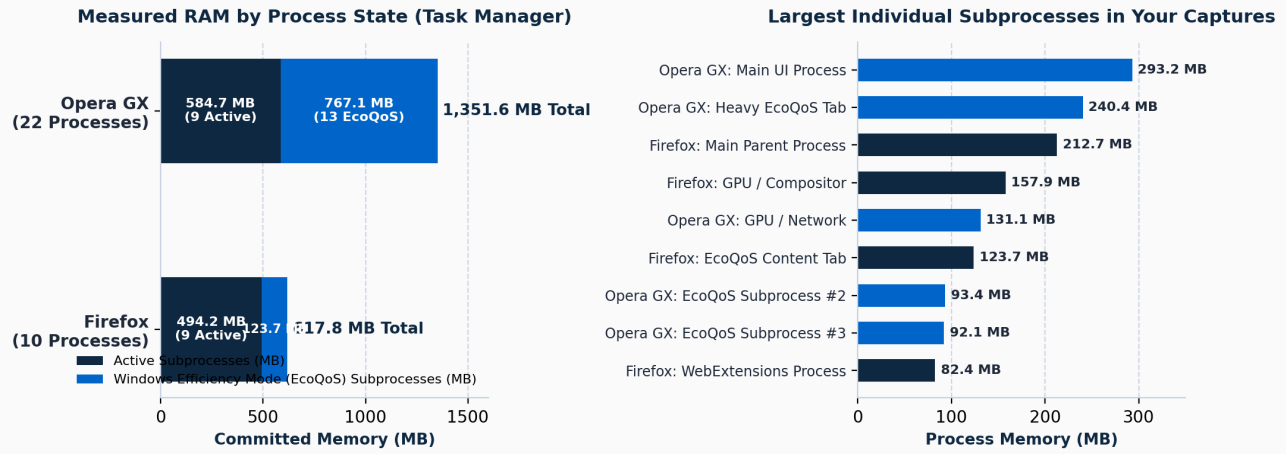


Figure 1: Exact subprocess memory breakdown from your Windows Task Manager captures. Opera GX retains 767.1 MB inside 13 Efficiency Mode processes because EcoQoS throttles CPU clocks without reclaiming virtual memory heaps.

Metric / Process Tier	Opera GX (Measured)	Stock Firefox (Measured)	Custom Gecko Build Target & Mechanism
Total Committed RAM	1,351.6 MB	617.8 MB	**280–360 MB Baseline** via Telemetry strip + Working-Set trim
Subprocess Count	22 Processes	10 Processes	**5–7 Processes** via consolidated `dom.ipc.processCount`
Active Process Footprint	584.7 MB (9 procs)	494.2 MB (9 procs)	**~250 MB** (Parent + WebRender GPU + Socket + 1–2 Active Tabs)
Efficiency Mode Footprint	767.1 MB (13 procs)	123.7 MB (1 proc)	**<15 MB** via DOM Discard + Win32 `EmptyWorkingSet` syscall
CPU Utilization (Idle/Light)	1.4% CPU	0.5% CPU	**0.0–0.2% CPU** via frozen background timers & native Rust adblock

2. Engine & Build Strategy: Gecko + Surfer Patch Engine (Rust & C++)

Choosing **Mozilla Gecko** over Chromium gives you three decisive structural advantages for a solo open-source project: (1) Gecko already ships with first-class Rust Components and Rust XPCOM interop alongside C++, (2) Gecko preserves the full blocking `webRequest` API so uBlock Origin runs at full strength (including CNAME uncloaking and HTML response filtering), and (3) Gecko's UI layer (HTML/XUL + CSS accelerated by Rust's WebRender GPU compositor) lets you build fluid, Opera GX-level animations without running a separate 293 MB UI helper tree.

Avoiding "Fork Fatigue" with Surfer + Mozilla Mach

Never clone and manually maintain a 35 GB fork of `mozilla-central`. Instead, model your repository after Zen Browser and LibreWolf by using Surfer (`@zen-browser/surfer`) alongside Mozilla's Mach build system:

- **Declarative Patch Overlay:** Your Git repository stays under 50 MB and contains only your custom Rust crates, C++ patches (`patches/*.patch`), browser chrome styling (`src/browser/themes/`), and hardened preferences.
- **Automated Upstream Rebasing:** Running `surfer download` fetches the exact upstream Firefox release source tarball; `surfer import` applies your git-format patches cleanly on top of Gecko, and `surfer build` invokes `./mach build` with Clang-CL and Rust `cargo` using Profile-Guided Optimization (PGO) and Link-Time Optimization (LTO).

```
my-gecko-browser/
├── surfer.json                 # Locks upstream Firefox version & branding metadata
├── mozconfig                   # Compiler flags (-O3, LTO, strip
telemetry/Pocket/CrashReporter)
├── patches/                    # Clean .patch diffs applied to Gecko C++/Rust source
│   ├── 01-necko-adblock-rust.patch
```

```
│   └── 02-win32-resource-governor.patch
├── src/
│   ├── rust-components/         # Custom Rust XPCOM crates linked into libxul
│   │   ├── resource_governor/   # Win32 Job Objects, EcoQoS & EmptyWorkingSet manager
│   │   ├── warp_socks_bridge/   # Embedded BoringTun/Wireproxy Cloudflare WARP tunnel
│   │   └── necko_shield/        # Native brave/adblock-rust HTTP channel interceptor
│   ├── browser/                 # Fluid vertical-tab UI, HUD limiter sliders, dark
theme
│   └── policies/                # distribution/policies.json & bundled uBlock Origin /
Dark Reader
```

3. Deep Subsystem Engineering Specifications

Subsystem A: True Hardware Resource Governor (Rust + Win32 APIs)

Opera GX's RAM limiter is largely advisory—if you set a limit without "Hard Limit" enabled, or if Chromium's V8 garbage collector refuses to release pages back to the OS, subprocesses stay bloated at 240+ MB. Your browser will implement a two-stage **Rust XPCOM Resource Governor** inside Gecko:

- **Stage 1 — OS Kernel Job Object Cap:** On Windows startup, the parent process creates a Win32 Job Object and configures `JOBOBJECT_EXTENDED_LIMIT_INFORMATION` via `SetInformationJobObject` using `JOB_OBJECT_LIMIT_JOB_MEMORY` and `JOB_OBJECT_LIMIT_PROCESS_MEMORY`, plus CPU rate control (`JOBOBJECT_CPU_RATE_CONTROL_INFORMATION`) wired directly to a live slider in your sidebar UI.
- **Stage 2 — Active DOM Unloading + Working-Set Purge:** Whenever a background tab is idle for more than 180 seconds (or total memory crosses 80% of your slider limit), the governor:
  1. Applies Windows Efficiency Mode (`SetProcessInformation` with `ProcessPowerThrottling`) to drop CPU scheduling to E-cores.
  2. Fires Gecko's internal `nsIMemoryReporterManager::MinimizeMemoryUsage` or `gBrowser.discardBrowser(tab)` to serialize session state (scroll position + form data) and destroy the DOM/JS heap.
  3. Calls Win32 `EmptyWorkingSet(hProcess)` and `SetProcessWorkingSetSizeEx` on the child PID, forcing Windows to reclaim the physical RAM pages immediately instead of leaving 767 MB cached in idle subprocesses.

Subsystem B: Dual-Layer Ad & Tracker Blocking (uBlock Origin + Native Rust)

Your browser combines the gold-standard extension capability of uBlock Origin with optional zero-memory native Rust network filtering:

- **Pre-Bundled Privileged uBlock Origin (Default On):** Package the latest signed `uBlock0@raymondhill.net.xpi` inside `distribution/extensions/` and enforce `"installation_mode": "force_installed"` in `distribution/policies.json`. Pre-populate its storage with EasyList, EasyPrivacy, uBlock filters, and Peter Lowe's list enabled on first boot.
- **Native Necko Rust Filter (`brave/adblock-rust`):** In your Firefox screenshot, the `WebExtensions` subprocess consumes **82.4 MB**. To shrink that overhead or provide a "Zero-Extension Ultra-Light Mode," embed Brave's `adblock-rust` crate directly into Gecko's Necko HTTP channel pipeline (`nsHttpChannel::AsyncOpen`). Requests matching EasyList/EasyPrivacy are aborted in compiled

Rust before leaving the parent process—using ~6 MB of static shared memory and zero JavaScript CPU cycles.

Subsystem C: Free Rootless Built-In VPN (Cloudflare WARP + BoringTun SOCKS5)

Building a free, high-speed VPN into an open-source browser poses two classic hurdles: commercial VPN gateways cost money, and OS-level WireGuard TUN drivers require Administrator privileges and hijack system-wide traffic outside the browser. You solve both using **Cloudflare WARP's free WireGuard network** paired with a **userspace WireGuard-to-SOCKS5 bridge**:

- **Automated Free Cloudflare WARP Provisioning (`wgcf` Protocol):** On first click of your browser's VPN toggle, a Rust background task generates an X25519 WireGuard keypair and sends a single HTTPS POST to Cloudflare's public consumer WARP registration endpoint (following the open-source `ViRb3/wgcf` specification). It receives a free client IPv4/IPv6 assignment and connects to `engage.cloudflareclient.com:2408` at zero cost and with no account signup.
- **Userspace Tunnel via BoringTun & Wireproxy:** Instead of installing a kernel TUN adapter, the browser spawns an embedded userspace tunnel (using Cloudflare's Rust `boringtun` cryptography and a userspace TCP/IP stack modeled on `wireproxy` / `wiresocks`) bound exclusively to `127.0.0.1:40000` (with random per-session port and SOCKS5 username/password auth so other local apps cannot piggyback).
- **Leak-Proof Gecko Proxy Binding:** Toggling the VPN dynamically switches Gecko's internal Necko preferences:

```
pref("network.proxy.type", 1);
pref("network.proxy.socks", "127.0.0.1");
pref("network.proxy.socks_port", 40000);
pref("network.proxy.socks_version", 5);
pref("network.proxy.socks_remote_dns", true);          // Prevents ISP DNS leaks
pref("media.peerconnection.ice.proxy_only", true);     // Prevents WebRTC IP leaks
pref("network.trr.mode", 3);                           // Enforces strict DNS-over-
HTTPS
```

Subsystem D: Universal Dark Mode & High-Fluidity Visual Chrome

To match Opera GX and Zen Browser's visual polish while keeping GPU memory under 120 MB:

- **120Hz WebRender Browser Chrome:** Build a collapsible vertical sidebar, workspace switcher, and real-time RAM/CPU/VPN telemetry pill using GPU-accelerated CSS transforms (`transform: translate3d(...)`, `will-change: transform`) rendered by Gecko's Rust WebRender engine.
- **Two-Tier Universal Dark Mode:**
  1. **Tier 1 (Zero CPU Cost — Native Dark Signals):** Lock `ui.systemUsesDarkTheme = 1` and `layout.css.prefers-color-scheme.content-override = 0` so every site supporting `@media (prefers-color-scheme: dark)` renders natively in dark mode at zero runtime cost, and style `about:blank` / page-load flashes in deep obsidian (`#0B0F17`) to eliminate white flashes between page loads.
  2. **Tier 2 (Smart Fallback for Legacy Light Sites):** Bundle Dark Reader via `policies.json` configured with **"Detect dark theme" enabled** (Dark Reader Dark Theme Detection) and default

to **Static / Filter+ mode** rather than Dynamic DOM-mutation mode on heavy web apps, preventing JavaScript style-recalculation lag.

4. Compile-Time & Runtime Hardening Matrix (`mozconfig` & Prefs)

Before writing a single line of custom C++, stripping Mozilla bloat and tuning Gecko's process allocator drops baseline RAM by ~180–220 MB compared to stock Firefox:

Configuration Layer	Flag / Preference Key	Value & Resource / Privacy Impact
Compile-Time (`mozconfig`)	`ac_add_options --enable-lto=cross` `ac_add_options --enable-release`	Enables Cross-Language Link-Time Optimization across Rust and C++ inside `libxul.dll`, shrinking binary size and improving instruction cache locality.

Compile-Time (`mozconfig`)	`MOZ_TELEMETRY_REPORTING=`
	`MOZ_SERVICES_HEALTHREPORT=`
	`MOZ_CRASHREPORTER=`
		Completely compiles out background telemetry ping threads, health reporting daemons, and crash dump monitors at build time.

Configuration Layer	Flag / Preference Key	Value & Resource / Privacy Impact
Process Pool (`vendor.js`)	`dom.ipc.processCount = 2` `dom.ipc.processPrelaunch.enabled = false`	Caps generic web content processes at 2 and disables speculative empty process pre-spawning, cutting idle subprocesses from 10 down to 5–6.
Memory Reclamation (`vendor.js`)	`browser.tabs.unloadOnLowMemory = true` `javascript.options.mem.gc_high_frequency_heap_growth_max = 120`	Makes SpiderMonkey's garbage collector compact heaps aggressively rather than waiting for 300% heap expansion.
Privacy & Anti-Fingerprinting	`privacy.resistFingerprinting = true` `network.http.referer.XOriginTrimmingPolicy = 2`	Spoofs hardware canvas/WebGL entropy, isolates cookies per top-level domain (Total Cookie Protection), and strips cross-origin tracking referers.

Bloat Removal (`vendor.js`)	`extensions.pocket.enabled = false`
	`browser.ml.enable = false`
	`browser.newtabpage.activity-stream.showSponsored = false`
		Disables Pocket integration, background local ML inference processes, and sponsored new-tab telemetry workers.

5. 16-Week Phased Execution Roadmap

This roadmap is structured so that you have a working, daily-driver custom browser by the end of **Week 3**, and then progressively layer in your Rust hardware limiters, native adblocking, and Cloudflare WARP VPN.

Phase & Timeline	Core Engineering Milestones	Verification & Target Metrics
Phase 1: Weeks 1–3 Foundation & Lean Gecko Build	• Set up MozillaBuild on Windows with Rust, Clang, and Surfer. • Configure `mozconfig` to strip Telemetry, Pocket, Normandie studies, and local ML workers. • Apply hardened `vendor.js` process-pool tuning and bundle uBlock Origin + Dark Reader via `policies.json`.	Compile clean 64-bit Windows build; verify ≤6 subprocesses and <360 MB RAM on initial boot with uBlock Origin active.
Phase 2: Weeks 4–7 Rust Resource Governor & Win32 Memory Trimmer	• Author `src/rust-components/resource_governor` using Rust XPCOM bindings and the Windows `windows-sys` crate. • Wire `SetInformationJobObject` hard RAM/CPU caps and automatic background tab DOM unloading + `EmptyWorkingSet` calls.	Verify that backgrounded tabs drop from ~200 MB to <15 MB within 3 minutes and hard RAM cap prevents total footprint from exceeding user slider.
Phase 3: Weeks 8–10 Native Rust Adblocker in Necko	• Integrate `brave/adblock-rust` into Gecko's C++ `nsHttpChannel` via `adblock-rust-ffi` or Rust XPCOM. • Add background filter-list updater with binary serialization (`Engine::serialize`) for instant 5ms startup loading.	Pass AdBlock Tester benchmarks at 98–100% score even with the WebExtensions process disabled ("Ultra-Lean Mode").
Phase 4: Weeks 11–13 Built-In Cloudflare WARP WireGuard VPN	• Implement automated Cloudflare WARP key registration (`wgcf` API flow) in Rust. • Embed userspace BoringTun / Wireproxy SOCKS5 daemon on `127.0.0.1` and wire one-click UI toggle with WebRTC/DNS leak protection.	Verify `https://1.1.1.1/cdn-cgi/trace` reports `warp=on` inside the browser while non-browser OS traffic remains untouched and zero admin prompts appear.

Phase & Timeline	Core Engineering Milestones	Verification & Target Metrics
Phase 5: Weeks 14–16 Fluid Visual Polish, Dark Mode & CI/CD	• Build custom vertical sidebar, workspace switcher, and Opera GX-style live RAM/CPU/VPN HUD panel in XUL/HTML/CSS. • Eliminate white flash on page transitions and configure GitHub Actions automated Windows installer (`.exe` / portable `.zip`) releases.	Steady 120 FPS UI animations, zero white flash on navigation, and automated reproducible open-source builds.